SOURCE-LEVEL GUIDE • TAILSCALE/TAILSCALE

Tailscale 客户端源码导读

Tailscale 把 WireGuard 变成“开箱即用”的私有网络。这份导读讲清 tailscaled 的结构、控制面协议、magicsock 怎么打洞和换路、DERP 为什么存在，以及一个数据包从 TUN 到对端的完整路径。

`tailscale/tailscale @ e906e69`

怎么读这份导读

这份导读写给有网络或系统背景、想读懂或定制 Tailscale 客户端的工程师。读完你应该能说清：`tailscaled` 由哪些子系统组成；节点怎么拿到网络图；两台机器在 NAT 后面怎么连上；一个包怎么走完全程；新功能该放哪。

全文顺序：

1. **全景**：问题、目标、限制，开源和闭源的边界。
2. **启动与结构**：`tailscaled`、`tsd.System`、LocalBackend。
3. **控制面**：登录、Noise、MapResponse。
4. **数据面**：`wgengine`、TUN、过滤器。
5. **magicsock**：打洞、路径选择。
6. **DERP**：最后的退路。
7. **周边**：DNS、netstack、tsnet、Tailnet Lock。
8. **功能模块化**：`feature/` 的方向。
9. **端到端**：一个 ping 的旅程。
10. **许可证、贡献与定制、源码索引**。

> **要点 · 固定版本**
>
> `tailscale/tailscale` @ `e906e69`（2026-11-05，版本 `1.105.0`）。我没有运行过这份代码，内容来自阅读源码和注释。

全景：问题、目标、限制

它解决什么

WireGuard 本身是一个优秀的数据面，但用它组网要自己解决：密钥分发、地址分配、NAT 穿透、访问控制。

环节	手动 WireGuard 的问题	Tailscale 的回答
密钥	每对节点手工交换公钥	协调服务器分发，私钥不离开设备
地址	手工规划	控制面分配 tailnet IP
NAT	两端都在 NAT 后就连不上	magicsock 打洞 + DERP 中继
身份	只有公钥	接入身份提供商
访问控制	自己写防火墙	控制面下发策略，每个节点本地过滤

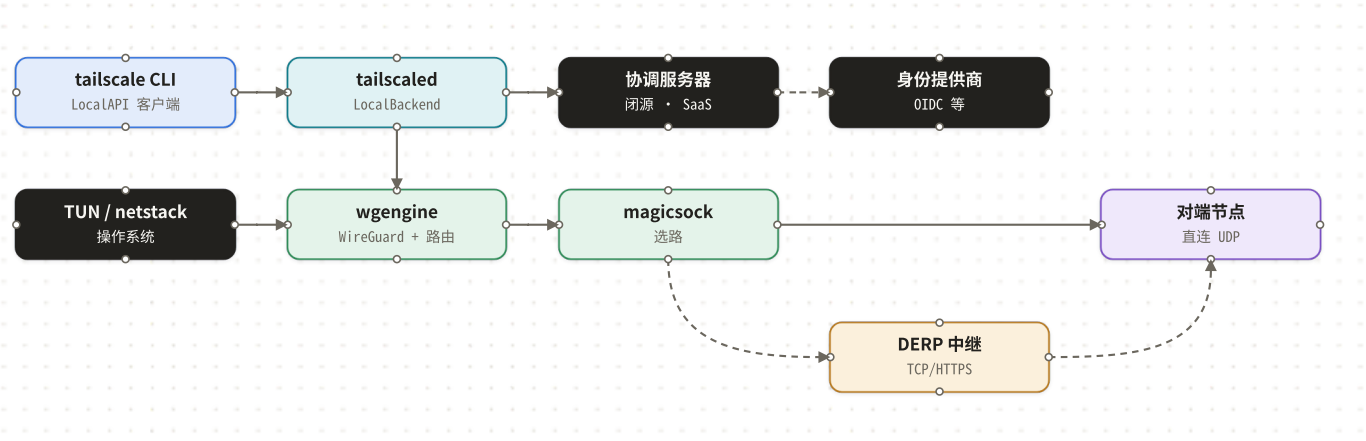
核心结构是**控制面和数据面分离**：协调服务器只交换元数据，用户流量点对点走，WireGuard 端到端加密。

要求 / 目标 / 限制

项目	内容
核心目标	让私有网络“装上就能用”
功能要求	任意 NAT 下连通；路由、出口节点、MagicDNS、SSH、Serve/Funnel
规模压力	大 tailnet 的 netmap 很大；小设备内存紧
外部限制	跨 Linux、macOS、Windows、BSD、iOS、Android；中间网络可能屏蔽 UDP
关键手段	用户态 WireGuard、magicsock、DERP、增量 netmap、模块化构建
许可证	BSD-3-Clause + 专利授权

总图

客户端和周边



深色不在这个仓库。协调服务器闭源；开源替代 Headscale 是第三方项目。DERP 服务器代码是开源的（`cmd/derper`）。

开源的边界

`README` 说得很清楚：

部分	状态
`tailscaled`、`tailscale` CLI、DERP	在这个仓库，开源
iOS、Android 使用的核心逻辑	在这个仓库
Android 应用	单独的开源仓库
macOS、iOS、Windows 的 GUI 外壳	闭源
协调服务器	闭源
WireGuard 实现	用 Tailscale 的 fork：`wireguard-go`

启动与结构

tailscaled

入口是 `main`，主要流程在 `run` 和 `getLocalBackend`。

`tryEngine` 决定数据面的形态：

模式	条件	行为
内核 TUN	默认	创建 tun、router、OS DNS 配置
`userspace-networking`	`--tun=userspace-networking`	不需要特权，用 netstack
TAP	`tap:` 前缀	二层设备

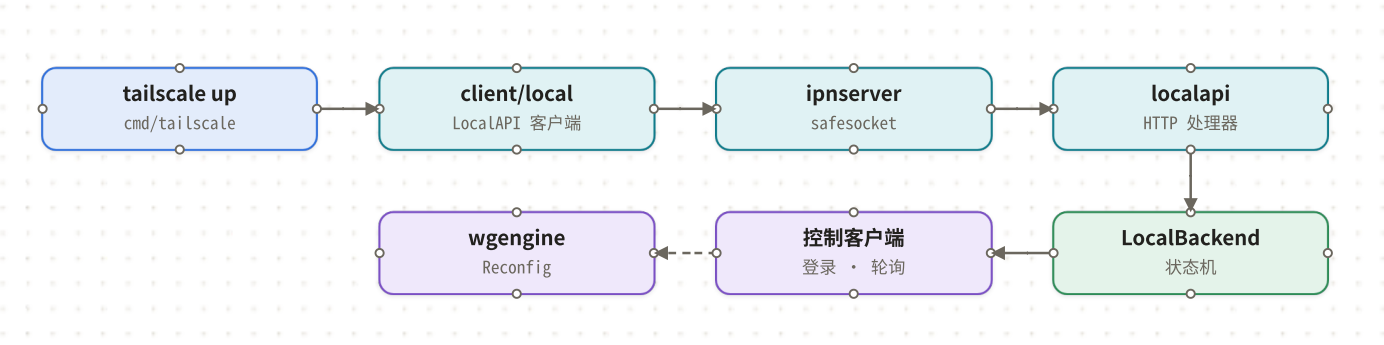
tsd.System

子系统靠 `tsd.System` 组装。包注释说明目标是统一 tailscaled、Windows 服务、macOS GUI、tsnet、wasm 的初始化，并提醒：这个包依赖几乎所有东西，所以不要把它传给不想依赖全世界的包。

LocalBackend

`ipn/ipnlocal/local.go` 有 9500 行，是客户端的大脑：连接控制客户端和 wgengine，维护状态机和用户偏好。所有东西都经过它，这也是 `feature/` 要解决的问题。

CLI 怎么控制 daemon



本地 API 走 unix socket 或 Windows 命名管道（`safesocket`），入口在 `serveHTTP`。客户端库见 `local.go`。

状态机

`nextStateLocked` 和 `enterStateLocked` 管理 `NoState`、`InUseOtherUser`、`NeedsLogin`、`NeedsMachineAuth`、`Stopped`、`Starting`、`Running`。状态定义在 `ipn/backend.go`。

收到新 netmap 时走 `setNetMapLocked`，然后 `authReconfig` 把它变成 wgengine 的配置。

> **注意 · 容易误解的地方**
>
> - 代码里的 `ipn` 没有明确的全称，`包注释`自己都在开玩笑：“Identified Private Network? IP Network?”
> - 大量 `*View` 类型是生成的只读视图（`cmd/viewer`），`tailcfg_view.go` 不要手改。

控制面

职责

`direct.go` 实现和协调服务器的通信，`auto.go` 负责自动重试和状态。

Noise

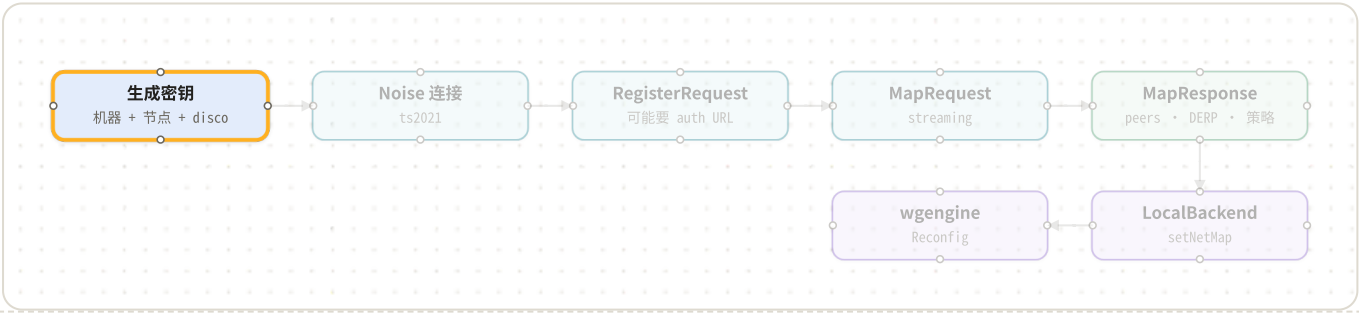
控制连接不依赖 TLS 的信任链。`ts2021` 是基于 Noise 的协议，建立在 `controlbase` 之上，拨号在 `controlhttp`，并且能穿过 HTTP 代理和只开 443 的环境。

两个核心请求

请求	类型	作用
登录	`RegisterRequest`	`TryLogin` 注册节点公钥
取网络图	`MapRequest` / `MapResponse`	`PollNetMap` 长轮询

`tailcfg` 定义了线上协议。`CapabilityVersion` 表示客户端能力，当前是 `150`。新旧客户端靠它协商，所以改协议时要加版本。

▶ **节点上线** 1 / 7



1. 密钥

节点有几把钥匙：机器密钥、节点（WireGuard）密钥、disco 密钥。私钥不离开设备，由 `types/key` 表示。

2. 连接

用 Noise 连到协调服务器，用它自己的公钥验证。

3. 登录

未授权时服务器返回 URL，用户在浏览器走身份提供商。

4. 轮询

`sendMapRequest` 发起 streaming 请求，带上自己的 endpoint 和 Hostinfo。

5. 接收

`MapResponse` 带来 peer 列表、DERP 地图、包过滤规则、DNS 配置。后续是增量更新，由 `map.go` 合并。

6. 进入后端

netmap 交给 `LocalBackend`。

7. 配置数据面

`authReconfig` 生成配置调用 `Reconfig`。

协调服务器不看你的流量

类比 · 电话簿，不是电话局

- **协调服务器** = 电话簿：记着每个人的公钥、地址、能打给谁。
- **MapResponse** = 定期寄来的新版电话簿。
- **WireGuard 隧道** = 两个人之间直接通话，电话簿公司不在线路上。
- **策略** = 电话簿上写着“不接 B 的电话”，而挂断动作发生在你自己家里（本地过滤器）。

所以协调服务器短暂宕机时，已有的连接继续工作，受影响的是新变化的分发。这一点是从架构推断的。

它只有公钥、地址、策略，没有私钥。Tailnet Lock 更进一步：节点要有本地信任签名才被接受，防止控制面被攻破时塞入恶意节点，实现在 `tka`。

数据面：wgengine

职责

`NewUserspaceEngine` 把这些东西组装到一起：TUN 包装、WireGuard 设备、magicsock、路由、DNS、过滤器。

组件	作用	位置
`tstun.Wrapper`	包装 TUN，在这里过滤和拦截	`wrap.go`
WireGuard	加解密	`wireguard-go` fork
`magicsock.Conn`	WireGuard 的 `conn.Bind`，负责选路	`Conn`
`router`	配置系统路由	`router.go`
`dns.Manager`	配置系统 DNS	`manager.go`
`filter.Filter`	有状态包过滤	`Filter`

`Reconfig` 的特点是比较差异：`engineChanged`、`routerChanged`、`dnsChanged`、`listenPortChanged`，没变化就提前返回。网络图频繁变化时这很重要。

关键巧思在 WireGuard 和 magicsock 的接口：**WireGuard 以为自己在往一个地址发包，实际地址是公钥**。magicsock 实现 `conn.Bind`，决定这个包这次走哪条路。WireGuard 完全不知道换路发生过，会话也不用重建。

`过滤器` 在每个节点本地执行控制面下发的规则，**入站方向**决定是否接受。

magicsock

这是项目最有特色的部分。包注释只有一句：“一个能在使用中改变通信路径的 socket，主动寻找最好的通信方式”（`magicsock.go`）。

候选地址

`determineEndpoints` 收集自己的候选：本地接口地址、STUN 发现的公网地址、端口映射（UPnP、NAT-PMP、PCP，`portmapper`）。`netcheck` 探测网络情况，包括 UDP 是否可用、NAT 类型、各 DERP 区延迟。

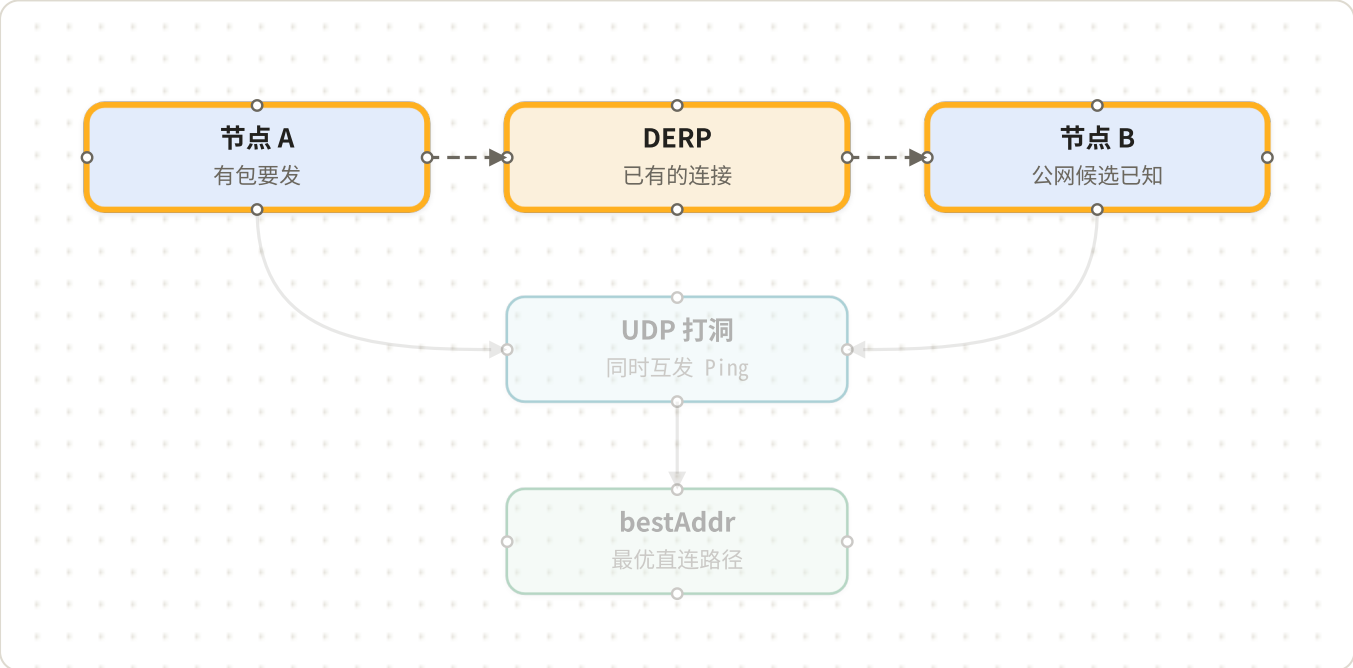
这些候选通过 `updateEndpoints` 报给控制面，再分发给 peer。

disco：路径探测

`disco` 是独立于 WireGuard 的小协议：头部是 magic `TS💬`、发送方 disco 公钥、nonce，内容用 nacl box 加密。消息类型有 Ping、Pong、CallMeMaybe。

为什么不直接用 WireGuard 探测？disco 用单独的密钥，可以在不建立 WireGuard 会话时探路。

▶ 两台 NAT 后的机器打洞 1 / 5



1. 先走中继

A 的第一个包可以经 DERP 发给 B，所以连接立即可用。

2. 探测

同时，`sendDiscoPingsLocked` 向 B 的所有候选地址发 disco Ping。A 自己的 NAT 因此打开映射。

3. CallMeMaybe

A 经 DERP 发 CallMeMaybe 给 B，内容是 A 的候选地址。B 收到后反过来向 A 发 Ping，打开自己的 NAT。这是打洞的关键：**两边要差不多同时发**。

4. 收到 Pong

一条路通了，`handlePongConnLocked` 记录延迟。

5. 换路

最优路径写入 `bestAddr`。之后的包直接走 UDP，WireGuard 无感知。

发送时选路

`addrForSendLocked` 很短，逻辑却是全部精华：

情况	行为
`bestAddr` 有效且未过期	只走 UDP
wireguard-only 节点	只有 UDP，没有 DERP
过期或没有	UDP 和 DERP **同时发**

`send` 在没有直连或信任过期时触发新一轮探测。过期时双发，保证切换期间不断流。`heartbeat` 在活跃时周期性刷新路径。

最后还有一招：如果不知道对方的 UDP 地址和 DERP home，就用对方上次联系我们的 DERP 区（`fallbackDERPRegionForPeer`）。

类比 · 同时打电话和发快递

- **WireGuard** = 寄件人，只写收件人姓名（公钥），不管地址。
- **magicsock** = 物流调度，决定走哪条路。
- **DERP** = 公司总部的中转站：慢但一定到。
- **disco Ping** = 派人试几条小路通不通。
- **CallMeMaybe** = 经总部捎个话：“我也在试，咱俩同时走，门就开了。”
- **路径过期时双发** = 同时走小路和中转站，避免换路时丢件。

寄件人从头到尾不知道包裹换过路。

换网络

切换 Wi-Fi 时，`linkChange` 响应 `netmon` 的事件，`ReSTUN` 重新探测。WireGuard 会话以公钥为身份，不绑定 IP，所以 TCP 连接能活过网络切换。

较新的 peer relay 用另一个节点做 UDP 中继，介于直连和 DERP 之间，见 `relaymanager.go` 和 `feature/relayserver`。

DERP

职责

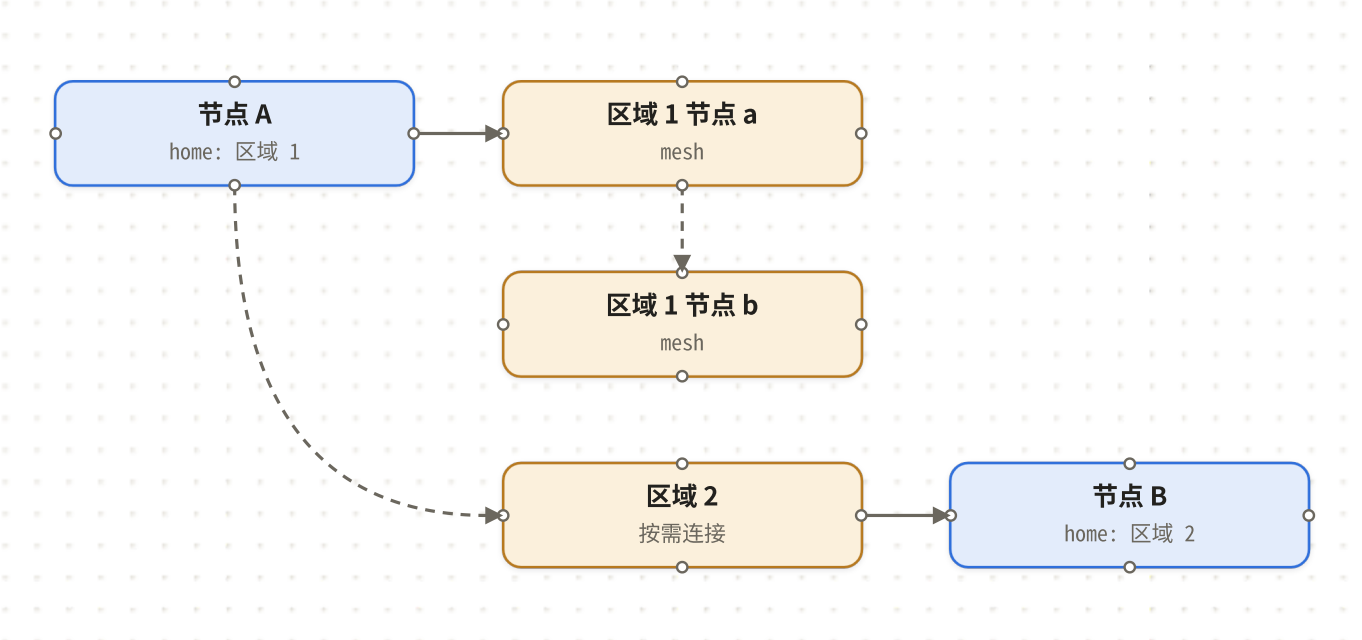
DERP 按 **curve25519 公钥寻址**，不是 IP。它中继两种包：disco 消息，以及 NAT 穿透失败时的 WireGuard 包（`derp/README.md`）。它是最后的退路，也是打洞的旁路信令。

DERP 走 TCP/HTTPS，所以 UDP 被屏蔽的网络里也能工作。

它看不到明文：中继的 WireGuard 包是端到端加密的。

区域和家

DERP 区域



每个节点选延迟最低的区作为 home，由协调服务器分享给所有 peer。发给 B 就连到 B 的 home。只有 home 连接要长期保持（`DERP Map`）。

设计取舍来自 README：

- 客户端自己按延迟选 home，所以不需要云负载均衡或 anycast，也不需要区域间路由。
- 同区节点 mesh，包只转发一跳。**区域之间没有路由**。
- 每区至少三个节点，不是为了选举，而是防止级联故障：两台各 51% 负载时，挂一台另一台也撑不住。

服务器在 `cmd/derper` 和 `derpserver`，还有 `xdp` 目录下的 XDP 实现。`stund` 提供 STUN。

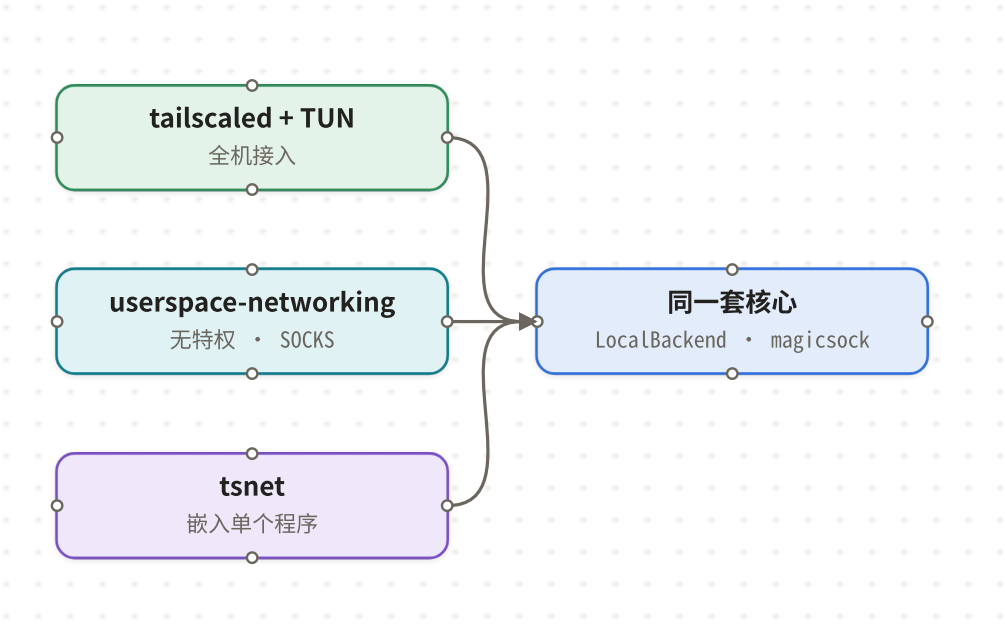
周边

功能	作用	位置
MagicDNS	tailnet 内域名，配置系统 DNS	`net/dns`
netstack	用户态 TCP/IP，基于 gVisor	`wgengine/netstack`
tsnet	把 Tailscale 节点嵌进 Go 程序	`tsnet`
Tailscale SSH	用 tailnet 身份认证 SSH	`ssh` 、 `feature/ssh`
Tailnet Lock	本地节点签名	`tka`
K8s operator	集群集成	`k8s-operator` 、 `cmd`
Taildrive	文件共享	`drive`

DNS 的难点是平台差异：`net/dns` 有 resolvconf、systemd-resolved、NetworkManager、Windows NRPT 等一堆实现，这里的文件大多是在和操作系统打交道。

tsnet 是最值得定制者关注的：一个 Go 程序不需要 TUN，不需要 daemon，自己就是 tailnet 上的节点。

三种接入方式



三种方式共用核心逻辑，差别在包从哪进出。

功能模块化

`feature/README.md` 讲了项目当前的重要方向：

客户端变大了，不是每个用户都要全部功能。一台几美元芯片的 IoT 设备不需要 Taildrop、WebDAV、ACME 或 SSH。所以**新功能优先放** `feature/<name>`，通过 hook 接入，不要塞进 `LocalBackend`。

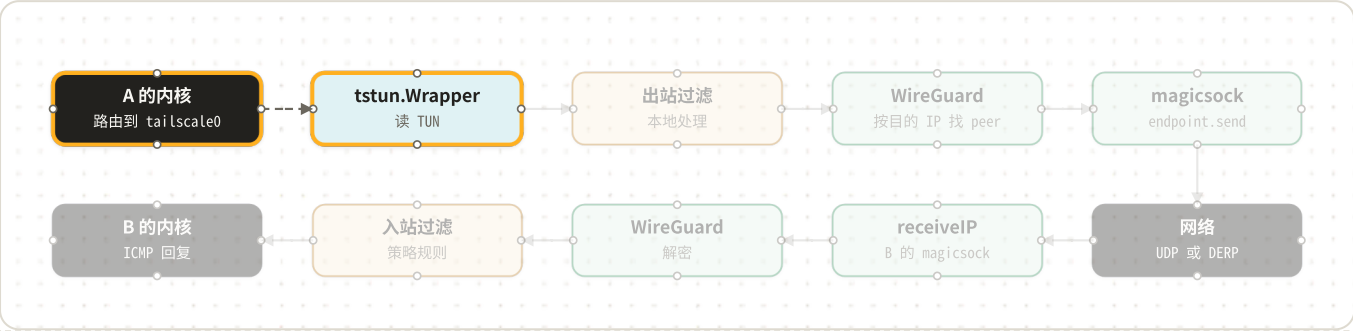
机制	说明
`feature/` 包	从自己的 `init` 注册
`ts_omit_*` 构建标签	去掉功能
`buildfeatures`	编译期常量，如 `HasDebug`、`HasNetstack`
`cmd/featuretags`	查看功能依赖
`cmd/omitsize`	算每个功能的体积

所以 `tailscaled.go` 里能看到 `hookNewNetstack.GetOk()` 这样的钩子。README 也承认大部分代码处于**半迁移状态**。贡献时要先读这份文档。

端到端：一个 ping

机器 A 上执行 `ping 100.64.0.2`，目标是 B。

▶ 一个 ICMP 包从 A 到 B 1 / 9



1. 进入 TUN

系统路由把 `100.64.0.0/10` 指向 `tailscale0`，路由由 `router` 配置。`tstun.Wrapper` 读到这个包。

2. 出站处理

`handleLocalPackets` 等钩子可以在这里处理或拦截。

3. 加密

WireGuard 按目的 IP 查 peer，加密。如果还没握手就先握手。

4. 选路

`Conn.Send` 调用 `endpoint.send`，`addrForSendLocked` 决定路径。

5. 传输

UDP 直发，或经 DERP，或同时。

6. 接收

B 的 `receiveIP` 区分 disco 和 WireGuard 包。

7. 解密

WireGuard 解密并验证公钥和 AllowedIPs。

8. 入站过滤

过滤器用控制面下发的规则判断，不允许就丢弃。

9. 投递

写回 TUN，B 的内核回 ICMP，反向走一遍。

开源许可证

部分	许可证	证据
仓库代码	BSD-3-Clause，Copyright 2020 Tailscale Inc	`LICENSE`
专利	额外的专利授权，类似 Go	`PATENTS`
依赖	各自许可证	`licenses/`
`tempfork/`	临时 fork 的第三方代码，各有许可	`tempfork`
WireGuard 商标	Jason A. Donenfeld 的注册商标	`README`

场景	允许吗	要做什么
自用	允许	无
提 PR	允许	**需要 DCO**：`Signed-off-by`
公开 fork	允许	保留版权；不要用 Tailscale 名义
发安装包	允许	用 `build_dist.sh` 带上版本信息
闭源嵌入	允许	保留声明，注意专利条款的终止条件
对外提供服务	允许	BSD 无网络条款，但协调服务器闭源，要用第三方实现或自己写

历史：GitHub API 显示 `LICENSE` 自 2020-02-05 进入这个仓库，之后三次修改的标题都是格式或版权声明调整，我没有逐条 diff。

> **不是法律意见**
>
> 这是工程师读代码时的理解。商标使用请看 Tailscale 自己的政策。

贡献与定制

开发环境

```
git clone https://github.com/tailscale/tailscale
cd tailscale
go install tailscale.com/cmd/tailscale{,d}
go test ./wgengine/magicsock/
make check
```

要求最新的 Go（`README`）。

项目规则

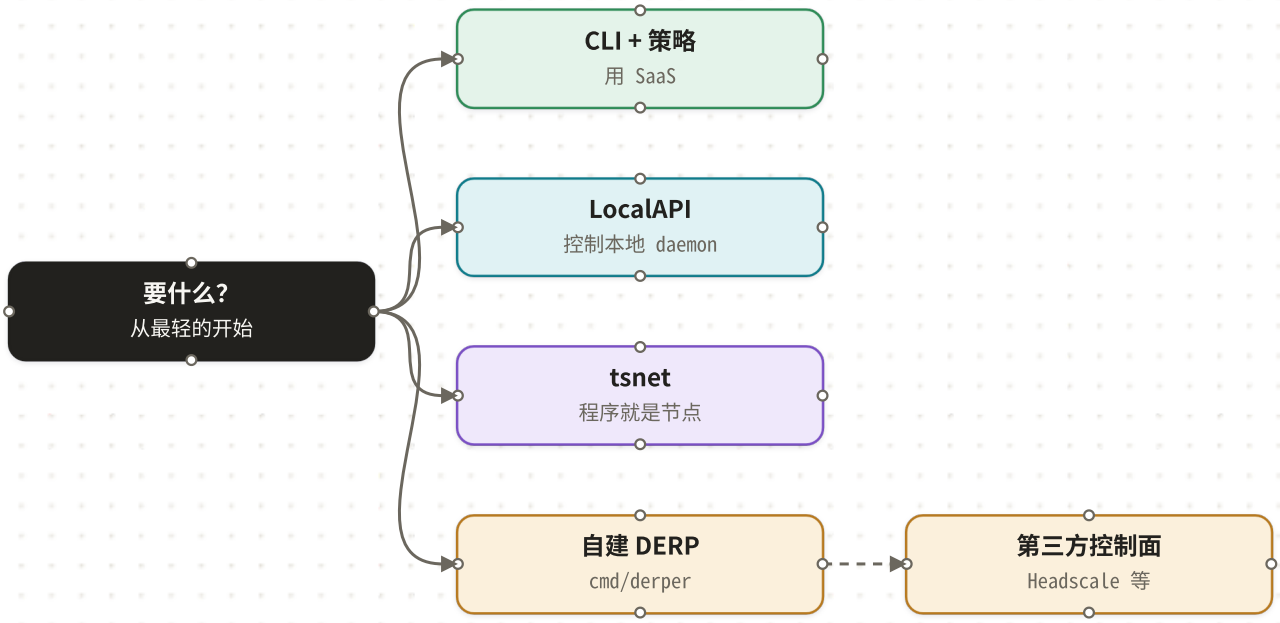
- 提交要有 DCO `Signed-off-by`，并引用 bug。
- 提交信息用 Go 和 Linux 风格：`wgengine/magicsock: fix foo`，**不用** Conventional Commits（`commit-messages.md`）。
- 新功能优先放 `feature/`。
- 协议变更要考虑 `CapabilityVersion`。
- 生成文件用 `viewer`、`cloner`。
- 改依赖关系要过 `depaware` 检查（`make check`）。

适合入手的方向

- 平台相关的 bug：DNS、路由。
- 把 `LocalBackend` 里的功能拆到 `feature/`。
- 测试：`tstest` 里有虚拟网络 `natlab`，可以模拟各种 NAT。
- 文档和诊断。

定制路线

想基于 Tailscale 做点什么



协调服务器闭源，自建完整网络要用第三方实现。仓库里的 `testcontrol` 只用于测试，不是生产控制面。

仓库里没有的东西

- **协调服务器**：闭源 SaaS。协议只能从 `tailcfg` 和客户端代码推断。
- **macOS、iOS、Windows GUI**：闭源外壳。
- **ACL 策略求值**：策略编译在服务器，客户端只收到编译后的 `FilterRule`。这一点是从客户端只收到 `FilterRule` 推断的。
- **Go fork**：发布版用 `tailscale/go`，但不是必需。

附录：源码索引

主题	链接
daemon 入口	`tailscaled.go`
CLI	`cmd/tailscale`
大脑	`local.go`
本地 API	`localapi`、`client/local`
控制协议	`tailcfg.go`、`direct.go`
引擎	`userspace.go`
路径	`magicsock.go`、`endpoint.go`
disco	`disco.go`
DERP	`README`、`cmd/derper`
TUN	`wrap.go`
过滤	`filter.go`
网络探测	`netcheck.go`
功能模块	`feature/README.md`
嵌入	`tsnet.go`
Tailnet Lock	`tka.go`